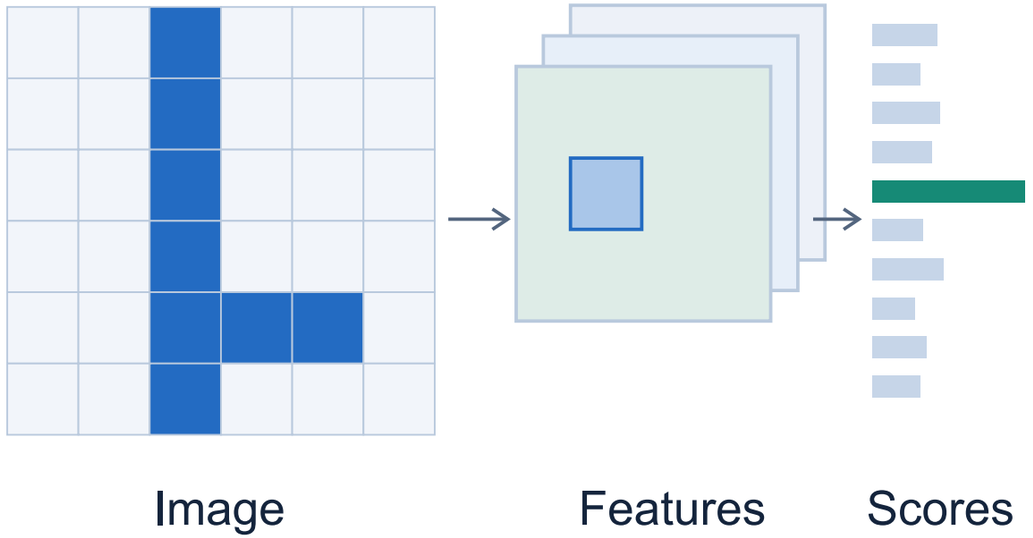
Convolutional Neural Networks

CS172 · Computer Vision I

Jiayuan Gu

ShanghaiTech University

Learning with spatial structure



From local evidence to a classifier

01 **Image structure**

Local connections and shared filters.

02 **Convolution**

Responses, channels, and gradients.

03 **Shape and context**

Stride, padding, and receptive fields.

04 **A complete CNN**

Layer shapes, cost, and implementation.

What you should be able to do

1. Explain locality and parameter sharing.
2. Calculate convolution outputs and gradients.
3. Derive tensor shapes and computational cost.
4. Trace receptive fields and translation behavior.
5. Implement and diagnose a small CNN.

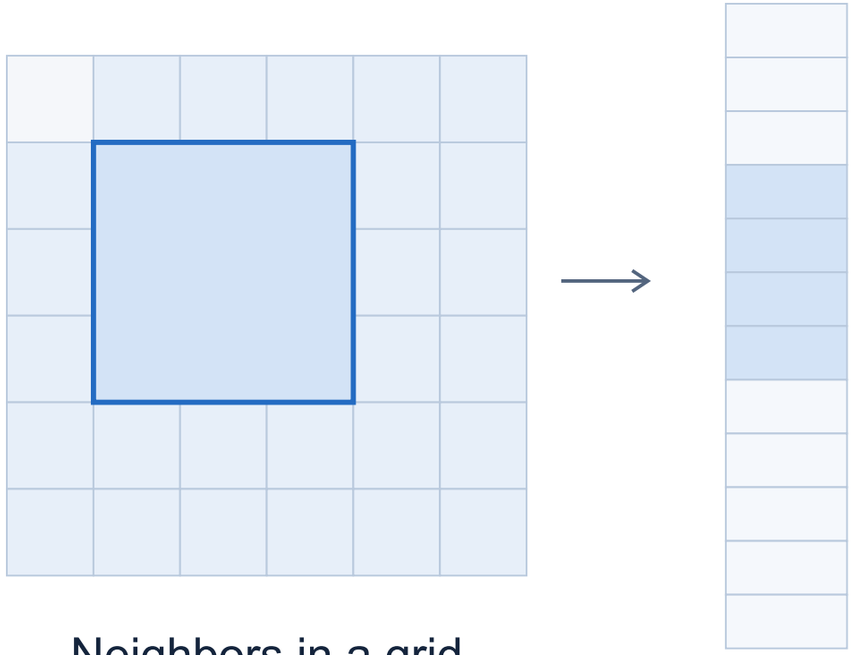
A dense layer has no built-in image geometry

An RGB image has shape $3 \times 32 \times 32$.

$$h = \mathrm{ReLU}(W\,\mathrm{vec}(X) + b)$$

Flattening retains every pixel value.

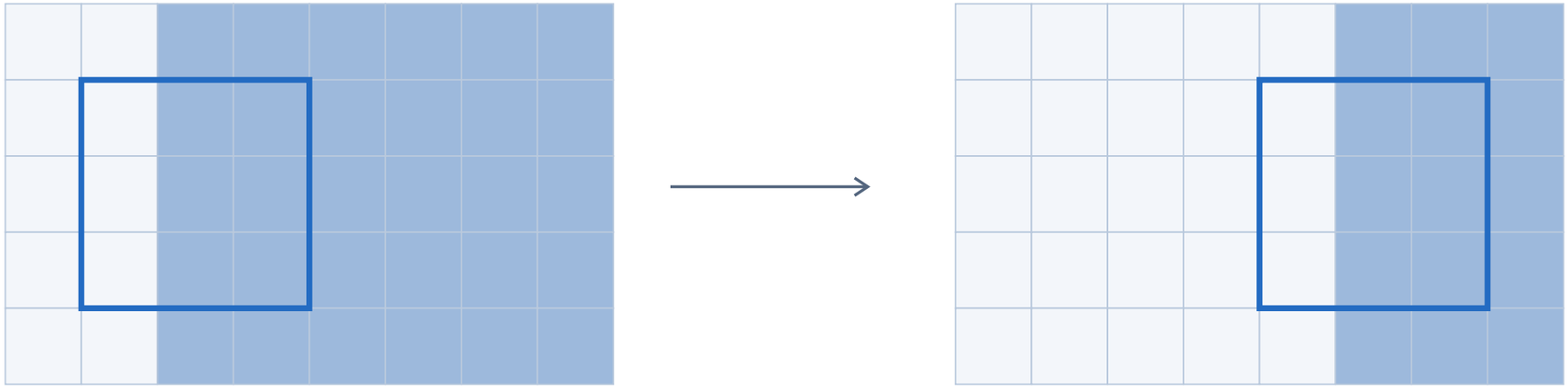
> A dense layer does not enforce which pixels are neighbors.



Neighbors in a grid

Vector

The same local pattern can occur anywhere



A local brightness transition

Same pattern, new position

> Reuse one local detector across the image.

Two architectural constraints

01

Local connectivity

Each output reads a small spatial neighborhood.

02

Parameter sharing

The same filter is applied at every spatial position.

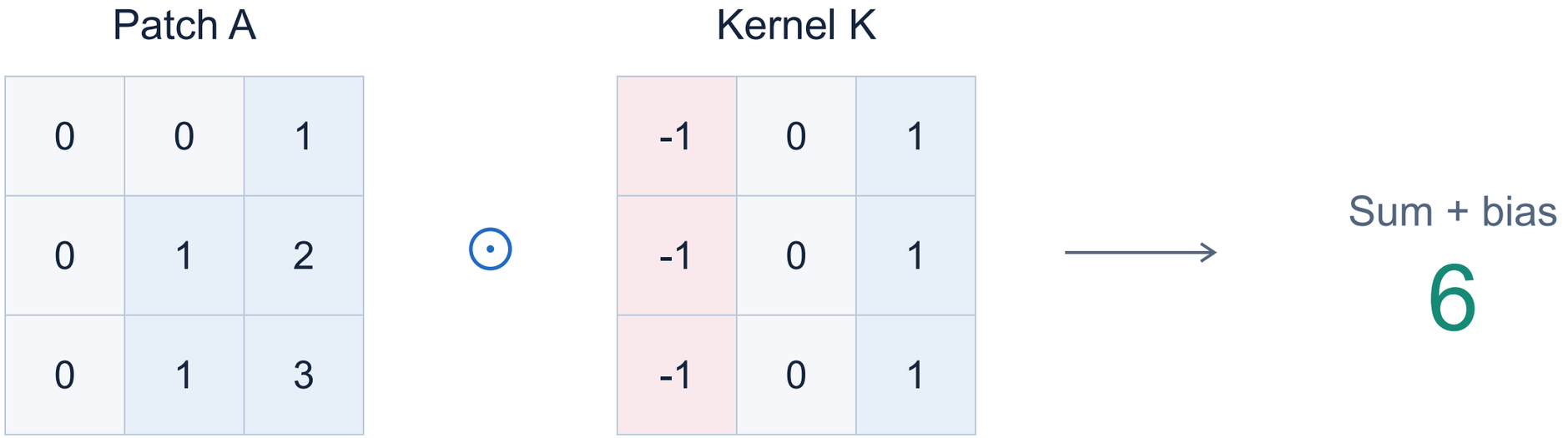
> **Inductive bias:** an assumption built into the model’s structure.

Compute one local response

A **kernel** (filter) is a small array of learned weights.

A patch’s dot product with the kernel, plus a learned offset (**bias**), gives one response.

One patch produces one number



$$z = \sum_{u=0}^{2} \sum_{v=0}^{2} A_{u,v} K_{u,v} + b$$

Here $b = 0$: $(1 - 0) + (2 - 0) + (3 - 0) = 6$.

Slide the same kernel across the image

Input X

0	0	1	1	0
0	1	2	1	0
0	1	3	2	0
0	1	2	1	0
0	0	1	0	0

Kernel K

−1 · left

-1	0	1
-1	0	1
-1	0	1

Output Z

6	2	-6
7	1	-7
6	1	-6

Selected: (0, 0)

response = 6

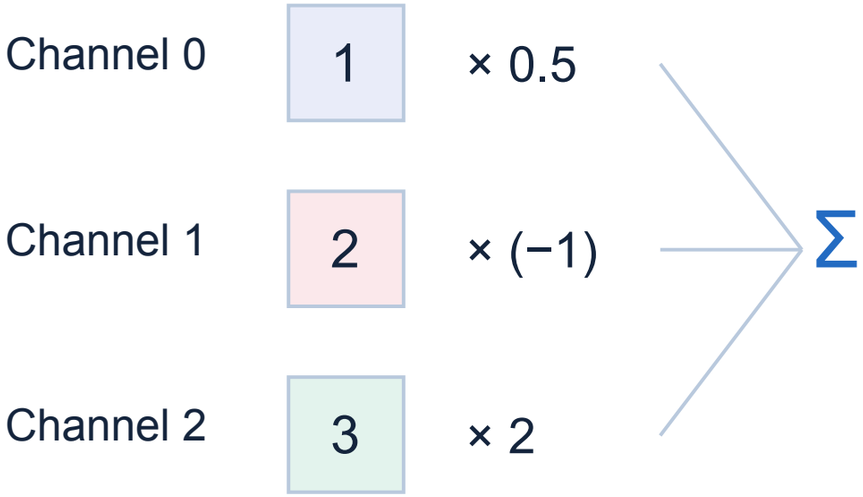
Input 5×5 · kernel 3×3
One-pixel steps · bias 0

Output position: **(0, 0)**

Row contributions: 1 + 2 + 3

Z[0, 0] = 6

Sum over every input channel



Bias: + 0.5 → output 5

An **input channel** is one image plane, such as red.

C_{in} counts input channels; C_{out} counts output channels.

> A standard filter sums across all input channels, then adds one bias.

A 1×1 example: $0.5(1) - 1(2) + 2(3) + 0.5 = 5$.

A filter bank creates output channels



Input X	**Weights** Θ	**Bias** b	**Output** Z
$N \times 3 \times 32 \times 32$	$8 \times 3 \times 3 \times 3$	8	$N \times 8 \times 32 \times 32$

Tensor: a multidimensional array. **Feature map:** one output channel. **Activations:** computed values.

NCHW: batch size, channels, height, width.

Write the operator precisely

$$Z_{n,o,i,j} = b_o + \sum_{c=0}^{C_{\text{in}}-1} \sum_{u=0}^{K-1} \sum_{v=0}^{K-1} \Theta_{o,c,u,v} \, \widetilde{X}_{n,c,iS+u,jS+v}$$

n: image · o: output channel

i, j: output row and column

c: input channel · u, v: kernel offsets

$\widetilde{X}$: input after zero padding

S: spatial stride · K: kernel side

One b_o is reused across the output map.

> Deep-learning “convolution” usually computes cross-correlation: the kernel is not flipped.

Shared weights collect gradients from all uses

$$x = [1, 2, 3], \quad \theta = [a, b], \quad z = [a + 2b,\ 2a + 3b]$$

Gradient arriving from later layers: $\partial L / \partial z = [4, 5]$.

$$\frac{\partial L}{\partial a} = 4(1) + 5(2) = 14 \qquad \frac{\partial L}{\partial b} = 4(2) + 5(3) = 23$$

> Backpropagation sums contributions from every use of a shared parameter.

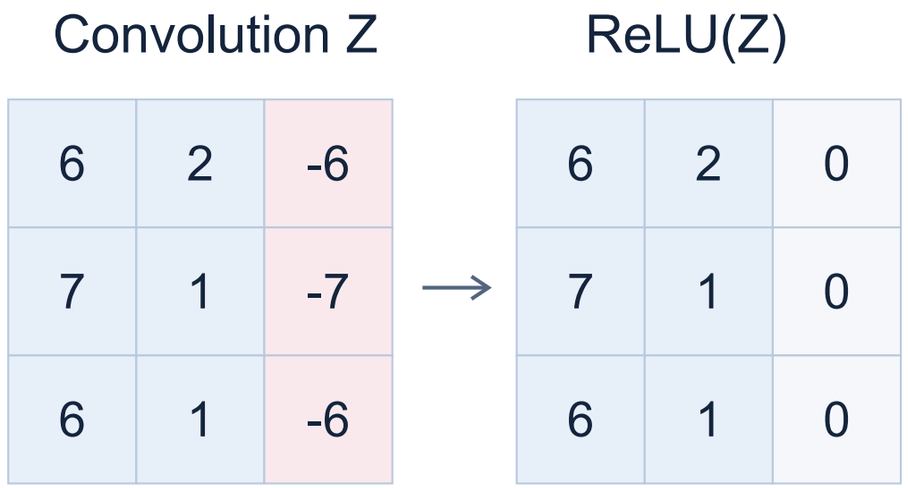
Nonlinearity makes stacked layers expressive

ReLU (rectified linear unit) clips negative values to zero.

$$A_{n,o,i,j} = \max(0, Z_{n,o,i,j})$$

ReLU preserves tensor shape.

> Without nonlinearities, stacked weighted sums still form one weighted sum.

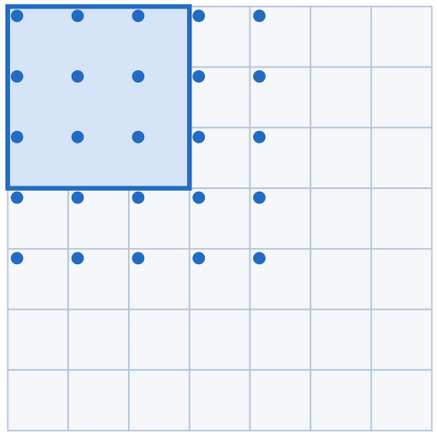


Same shape: 1 × 3 × 3

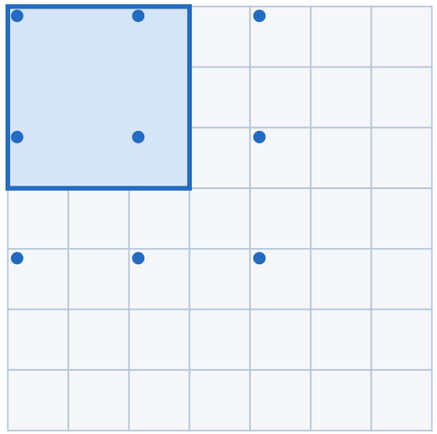
Control shape and cost

Kernel size and step size determine the output shape and the amount of computation.

Stride sets the step size



Stride 1: 5 × 5 output

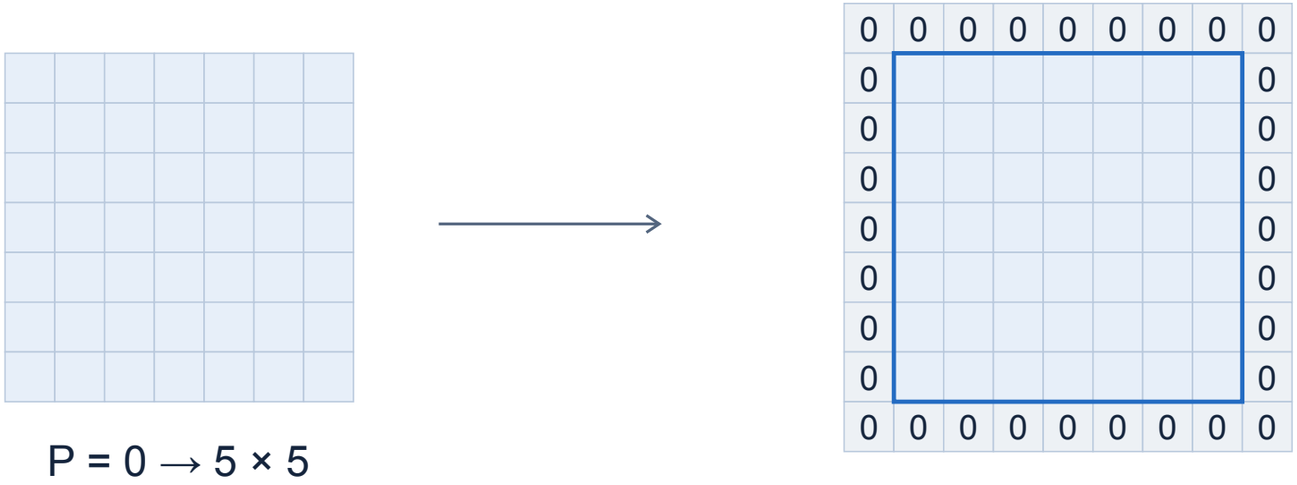


Stride 2: 3 × 3 output

Stride S: the number of input pixels the kernel moves per step.

$S = 1$: every starting position. $S = 2$: every other position.

Padding adds a boundary convention



Zero padding: add P rows or columns of zeros on each side.

For odd K, stride 1: $P = (K - 1)/2$ preserves spatial size.

Count valid kernel placements

$$H' = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1, \qquad W' = \left\lfloor \frac{W + 2P - K}{S} \right\rfloor + 1$$

Floor $\lfloor \cdot \rfloor$: round down to an integer.

Exactly divisible

$$H = 7,\ K = 3,\ S = 2,\ P = 0$$

Starts $0, 2, 4 \to H' = 3$.

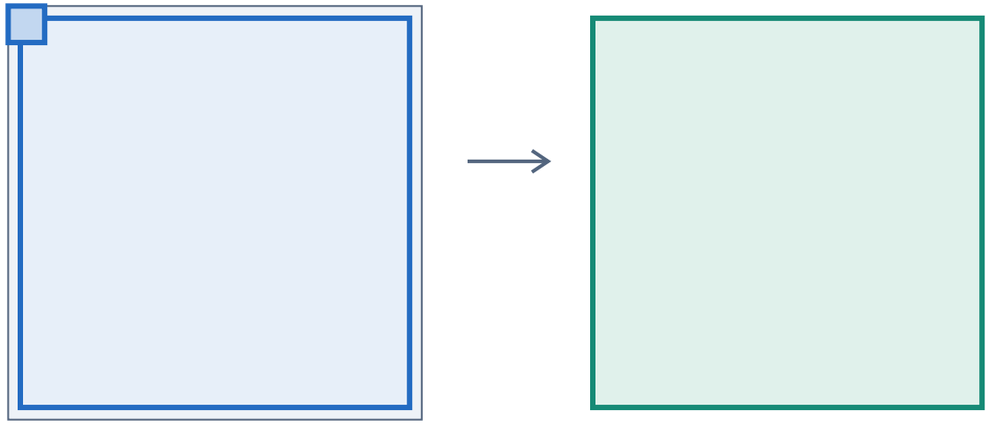
A trailing strip remains

$$H = 8,\ K = 3,\ S = 2,\ P = 0$$

Still starts $0, 2, 4 \to H' = 3$.

See how each setting changes the output

Input: 3 × 32 × 32 Output: 8 × 32 × 32



Input side $H = W$	Kernel K
32	3
Stride S	Padding P
1	1

Output: 8 × 32 × 32

224 parameters · 8,192 output values per image

H′ = floor((32 + 2·1 − 3) / 1) + 1

= 32 positions per spatial axis

Fixed $C_{\text{in}} = 3$, $C_{\text{out}} = 8$, one image, bias enabled.

Work through a convolutional layer

Input: $N \times 8 \times 16 \times 16$

16 filters · $K = 3,\ S = 2,\ P = 1$ · bias enabled

Weights: $16 \times 8 \times 3 \times 3$

Output: $N \times 16 \times 8 \times 8$

Parameters: $16(8 \cdot 9 + 1) = 1{,}168$

> Batch size N changes the number of outputs, not the learned weights.

Sharing separates parameters from output size

Same input $3 \times 32 \times 32$ and output $8 \times 32 \times 32$.

Connections	Parameters, including bias
Fully connected	$8192(3072 + 1) = 25{,}174{,}016$
Local 3×3, no sharing	$8192(27 + 1) = 229{,}376$
Local 3×3, shared	$8(27 + 1) = 224$

$$\#\text{Conv parameters} = C_{\text{out}}(C_{\text{in}}K^2 + 1)$$

All three layers produce 8,192 output values. Padding 1 for local layers.

Count the multiply–accumulate operations

MAC = multiply–accumulate. Multiply two numbers, then add the product to a running sum.

$$a \leftarrow a + xw$$

Each output uses $C_{\text{in}} K^2$ MACs.

$$\#\text{MACs} = H' W' C_{\text{out}} C_{\text{in}} K^2$$

One image: $3 \times 32 \times 32 \to 8 \times 32 \times 32$, $K = 3$.

$$32 \cdot 32 \cdot 8 \cdot 3 \cdot 9 = 221{,}184 \text{ MACs}$$

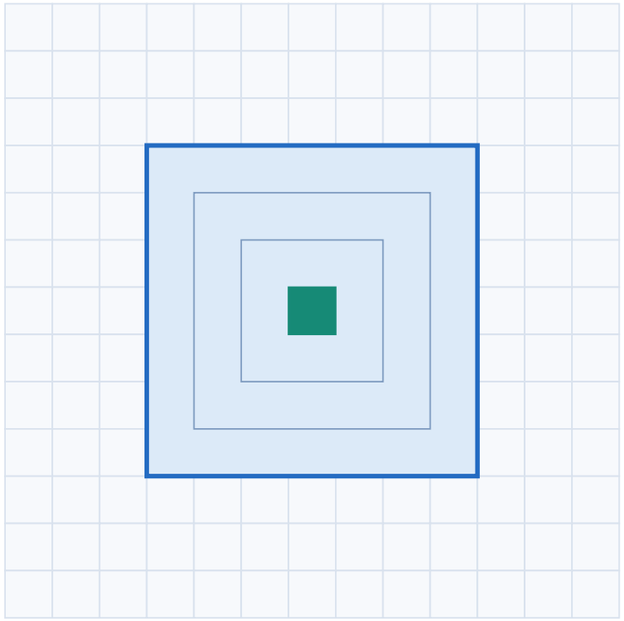
224 learned parameters; 221,184 MACs per image. Bias additions are excluded.

Let local evidence cover larger regions

Stacking layers combines local evidence. Downsampling reduces spatial resolution so later filters cover larger input regions.

Stacked layers expand the receptive field

Original input · field 7 × 7



Green: tracked output center

Receptive field: the input region that can affect one output.

Every layer: 3×3, stride 1; track an output away from the border.

Number of convolution layers: **3**

$$r_L = 1 + L(K - 1)$$

Receptive field: 7 × 7 input pixels

Downsampling makes later context grow faster

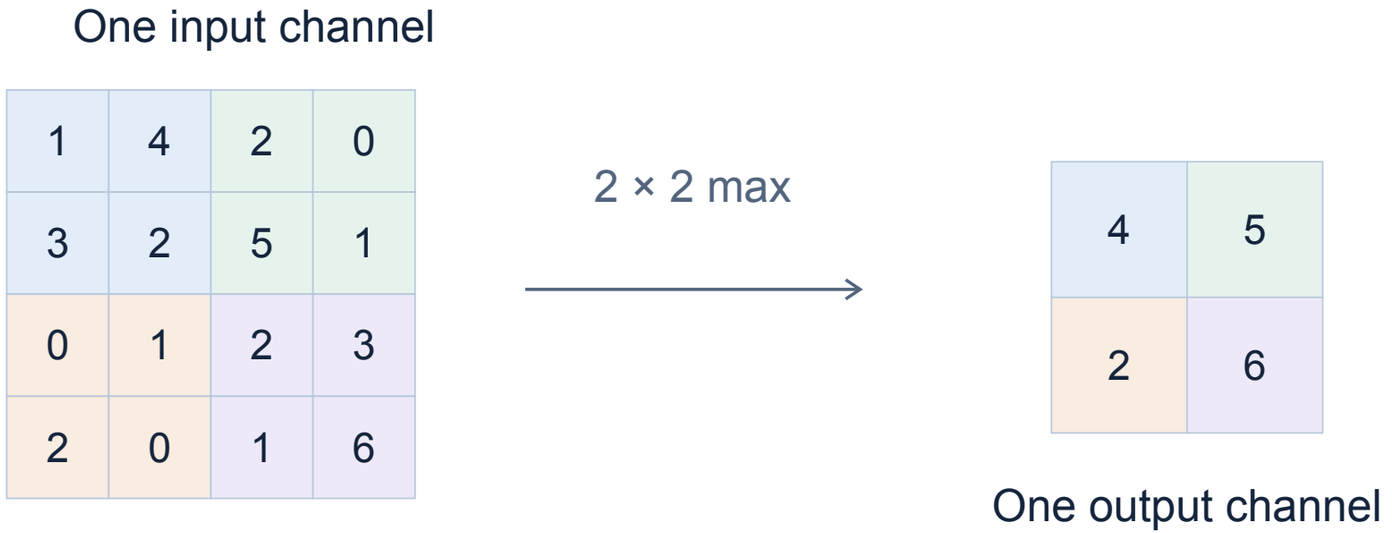
r_ℓ: receptive-field side; j_ℓ: spacing between output centers in input pixels.

$$r_\ell = r_{\ell-1} + (K_\ell - 1) j_{\ell-1}, \qquad j_\ell = S_\ell j_{\ell-1}$$

Layer	K, S	r	j
Input	—	1	1
Conv → max pool	3,1 → 2,2	3 → 4	1 → 2
Conv → max pool	3,1 → 2,2	8 → 10	2 → 4

> The second 3×3 convolution adds four input pixels to the field's side.

Pooling summarizes each channel separately



Max: keep the strongest response.

Average: summarize the mean response.

$K = 2, S = 2$: halve each spatial side.

No learned weights; channel count stays fixed.

Equivariance is different from invariance

T_δ shifts an image or feature map by δ pixels.

Equivariance

$$f(T_\delta X) = T_\delta f(X)$$

Shift the input; the feature map shifts.

Stride-1 convolution on an infinite grid, or away from affected boundaries.

Invariance

$$f(T_\delta X) = f(X)$$

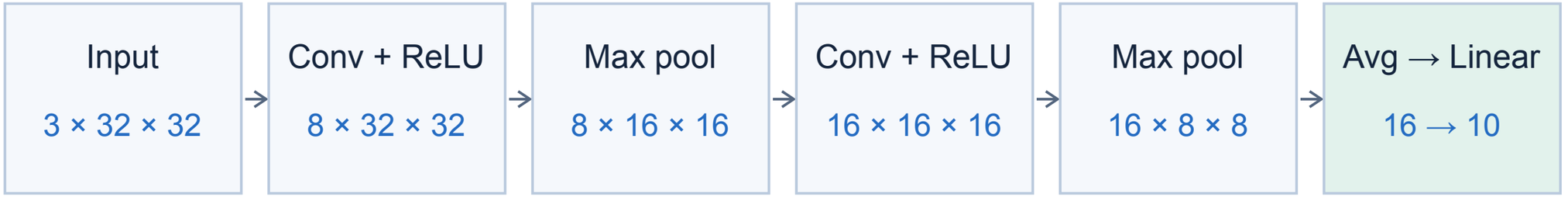
Shift the input; the output stays the same.

An additional property, not implied by sharing.

Max pool, $K = 2, S = 2$: $[0, 1, 0, 0] \mapsto [1, 0]$

Shift right one pixel: $[0, 0, 1, 0] \mapsto [0, 1]$.

Assemble a small image classifier



Convolution: K = 3, S = 1, P = 1 · Max pool: K = 2, S = 2

Global average pooling: average each final 8×8 map into one number.

> The **classification head** maps 16 features to 10 **logits** (unnormalized class scores).

Check every layer before training

Layer	Output (batch axis omitted)	Parameters
Input	$3 \times 32 \times 32$	0
Conv $3 \to 8$, $K = 3, P = 1$ + ReLU	$8 \times 32 \times 32$	224
Max pool $K = 2, S = 2$	$8 \times 16 \times 16$	0
Conv $8 \to 16$, $K = 3, P = 1$ + ReLU	$16 \times 16 \times 16$	1,168
Max pool $K = 2, S = 2$	$16 \times 8 \times 8$	0
Global average $\to$ Linear $16 \to 10$	$16 \to 10$	170

> Total: **1,562 parameters**, including biases. Conv stride is 1.

Express the same architecture in PyTorch

```
import torch
from torch import nn

model = nn.Sequential(
    nn.Conv2d(3, 8, 3, padding=1), nn.ReLU(),
    nn.MaxPool2d(2),
    nn.Conv2d(8, 16, 3, padding=1), nn.ReLU(),
    nn.MaxPool2d(2),
    nn.AdaptiveAvgPool2d(1), nn.Flatten(1),
    nn.Linear(16, 10),
)
x = torch.randn(4, 3, 32, 32)
assert model(x).shape == (4, 10)
```

Train the logits with the familiar objective

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
criterion = nn.CrossEntropyLoss()
model.train()

# One synthetic batch: class indices 0, 1, 2, 3
target = torch.tensor([0, 1, 2, 3], dtype=torch.long)
optimizer.zero_grad()
logits = model(x)              # shape: (4, 10)
loss = criterion(logits, target)
loss.backward()
optimizer.step()
```

CrossEntropyLoss takes raw class scores and handles probability normalization internally.

Repair two common tensor mismatches

Channels in the wrong axis

```
images.shape  # (4, 32, 32, 3)
nn.Conv2d(3, 8, 3)(images)
```

Conv2d expects channels in axis 1.

Move the last axis to axis 1:

```
images.permute(0, 3, 1, 2)
# (4, 3, 32, 32)
```

Too many features for the head

```
features.shape  # (4, 16, 8, 8)
features.flatten(1)
# (4, 1024)
```

A 16-feature head cannot take 1,024 values.

Average each map before flattening:

```
nn.AdaptiveAvgPool2d(1)
# (4, 16, 1, 1) → (4, 16)
```

Architectural assumptions have limits

- **Position:** global averaging discards spatial arrangement.
- **Detail:** downsampling can remove small objects and thin structures.
- **Variation:** sharing alone does not handle rotation, scale, or changed imaging conditions.
- **Influence:** pixels inside a receptive field need not affect the output equally.

Connect structure, geometry, and learning

1. **Structure:** local connections and shared filters encode reusable evidence.
2. **Computation:** each response sums spatial and channel products.
3. **Geometry:** kernel, stride, and padding determine output shape.
4. **Context:** depth and downsampling expand receptive fields.
5. **Learning:** shared gradients and familiar losses train the whole classifier.

Check your understanding

1. Input $8 \times 32 \times 32$; 16 filters, $K = 3, S = 2, P = 1$.
   Output shape and parameter count?

   $16 \times 16 \times 16$; $16(8 \cdot 9 + 1) = 1{,}168$ parameters.

2. Two 3×3, stride-1 convolutions: receptive field?

   5×5 in the original input, for an output away from the border.

3. Does weight sharing imply translation invariance?

   No. A shift can move the feature map; strided pooling can change which window captures a feature.

Reading and further study

- Stanford CS231n · Lecture 5

  Convolution, spatial dimensions, computational cost, and pooling.

- Dive into Deep Learning · §§7.1–7.5

  Locality, sharing, padding, channels, and pooling.

- PyTorch · Conv2d and CrossEntropyLoss

  Input/output shapes and loss-function usage.

- Supplementary: Araujo, Norris & Sim · Distill (2019)

  Receptive-field derivations and alignment.